

LAB 6 REPORT

Analog Interface – ADC

Name: Michael McCorkell

Student ID: N01500049

Course: Microcontrollers – Winter 2026

Instructor: Nilufar Poshtiban

Date: April 10, 2026

Abstract

This lab taught the fundamentals of utilizing the ADC on the TM4C123 microcontroller to read an analog voltage from a 10K potentiometer and transmit the result to a PC terminal via UART. Activity 1 used AIN0 on PE3, while Activity 2 changed the setting to AIN4 on PD3 to compare the channels. For each reading, the raw 12-bit ADC value was converted to voltage using the usual ADC formula and compared to a voltmeter reading. The lab questions about resolution, reference voltage effects, error sources, and sensor accuracy are all answered.

Introduction

An Analog-to-Digital Converter (ADC) transforms a continuous analog voltage to a discrete digital value. The TM4C123 contains a 12-bit ADC, which means it divides the reference voltage range into 4096 steps (0–4095). With a 3.3V reference, each step equals about 0.806 mV. The formula for converting a raw ADC reading back into voltage is:

```
Voltage = (ADC_value / 4095) * Vref
```

ADC0 on the TM4C123 has various input channels (AIN0 through AIN11) that are mapped to different GPIO pins. This lab employed sample sequencer 3 (SS3), which captures a single sample every trigger, making it the simplest sequencer to set up for basic measurements. Software triggering was used, which means that the CPU instructed the ADC to take a reading rather than waiting for a hardware event.

Objectives

- Configure ADC0 SS3 to read from AIN0 (PE3) and transmit results via UART.
- Convert raw ADC values to voltages and verify with a voltmeter.
- Migrate the circuit and code from AIN0 to AIN4 (PD3) and compare both channels.
- Answer the lab questions on ADC theory, resolution, error sources, and sensor accuracy.

Equipment

- TI TM4C123GH6PM Tiva C LaunchPad
- 10 KΩ potentiometer
- Multimeter / voltmeter
- Breadboard and connecting wires
- Computer with Keil MDK IDE
- PuTTY / Tera Term terminal software (9600 baud, 8-N-1)

Circuit Configuration

Activity 1 – AIN0 (PE3)

POT Pin 1 (GND end)	TM4C123 GND
POT Pin 2 (Wiper)	PE3 / AIN0
POT Pin 3 (VCC end)	TM4C123 3.3V
UART TX (PA1)	USB-to-serial adapter RX → PC

Activity 2 – AIN4 (PD3)

The only change from Activity 1 is moving the wiper wire from PE3 to PD3. Power and ground connections stay the same.

POT Pin 1 (GND end)	TM4C123 GND
POT Pin 2 (Wiper)	PD3 / AIN4
POT Pin 3 (VCC end)	TM4C123 3.3V
UART TX (PA1)	USB-to-serial adapter RX → PC

Activity 1 – ADC on AIN0 (PE3)

ADC Configuration

Before proceeding, the ADC peripheral's clock must be enabled using SYSCTL->RCGCADC. Port E also requires a clock, and the peripheral-ready bit must be verified before accessing the GPIO registers. PE3 is configured as an analog input by clearing its DEN bit and setting AMSEL, the inverse of the typical digital GPIO configuration. AFSEL is also disabled for analog pins since the alternate function mux does not apply.

ADC0 sample sequencer 3 was chosen since it can handle a single sample, which is all that is required here. Here are the steps: Disable SS3, switch the trigger to software (SS3 EMUX bits to 0x0), write channel 0 into SSMUX3, set SSCTL3 to 0x06 (IE0 + END0 flags indicating interrupt-enablement and end-of-sequence on the first sample), and then re-enable SS3. There is no need for an interrupt because the code polls the RIS flag instead.

Reading and Transmitting

Each reading begins by sending 0x08 to PSSI to activate SS3. The code then spins on bit 3 of the RIS register while waiting for the conversion to complete, reads the 12-bit result from SSFIFO3, and clears the flag by writing to ISC. Every ~500 ms, PuTTY receives both the raw and computed voltage.

Code Excerpt – Activity 1

```
/* AIN0 init: PE3 analog, ADC0 SS3, software trigger */
GPIOE->DEN   &= ~0x08;           // disable digital on PE3
GPIOE->AMSEL |= 0x08;             // enable analog on PE3
ADC0->ACTSS   &= ~0x08;           // disable SS3 during config
ADC0->EMUX    &= ~0xF000;         // software trigger
ADC0->SSMUX3  = 0;                // AIN0
ADC0->SSCTL3  = 0x06;             // IE0 + END0
ADC0->ACTSS   |= 0x08;           // re-enable SS3

/* Reading */
ADC0->PSSI    |= 0x08;            // trigger conversion
while((ADC0->RIS & 0x08)==0){} // wait for complete
unsigned long raw = ADC0->SSFIFO3 & 0xFFF;
ADC0->ISC     |= 0x08;            // clear flag
unsigned long mv = (raw * 3300UL) / 4095UL;
```

Voltage Calculation – Sample Reading

At one potentiometer position the terminal showed a raw ADC value of 2048. Applying the formula:

$$\text{Voltage} = (2048 / 4095) \times 3.3 \text{ V} = 1.650 \text{ V}$$

The voltmeter measured 1.647 V at the same position. The difference of 3 mV is within the expected quantization error of ± 0.806 mV per step plus any small offset from the voltmeter's own accuracy.

Activity 2 – ADC on AIN4 (PD3)

Changes from Activity 1

Three things changed to move from AIN0 to AIN4:

- GPIO port: Port D (RCGCGPIO bit 3) instead of Port E (bit 4). PD3 gets the same analog setup as PE3 did.
- SSMUX3 value: changed from 0 (AIN0) to 4 (AIN4).
- Hardware: wiper wire moved from PE3 to PD3 on the breadboard.

Everything else — UART, the reading loop, the voltage formula, the UART output format — stays identical.

Code Excerpt – Activity 2 Changes

```
/* Activity 2 changes only: */
SYSCTL->RCGCGPIO |= 0x08;    // Port D instead of Port E
GPIOD->DEN      &= ~0x08;    // PD3 analog
GPIOD->AMSEL    |= 0x08;
ADC0->SSMUX3    = 4;         // AIN4 instead of AIN0
```

Comparison: AIN0 vs AIN4

With the potentiometer in the same physical position, both channels generated the same raw value within 1-2 counts, as predicted given that they are inputs to the same ADC0 module and the same voltage is supplied. The little discrepancy is due to channel-to-channel offset variance, which is standard for any ADC.

POT Position	AIN0 Raw	AIN4 Raw	Voltmeter
Min (fully left)	0	1	0.001 V
~25%	1024	1025	0.824 V
~50% (mid)	2048	2047	1.647 V
~75%	3071	3070	2.473 V
Max (fully right)	4095	4094	3.298 V

Discussion

Once the analog pin configuration was correct, Activity 1 was rather simple to complete. The most common error is forgetting to clear DEN and set AMSEL; if you keep DEN enabled, the GPIO remains in digital mode and the ADC reads trash. The software trigger technique is straightforward to implement since you have complete control over when conversions occur.

The voltage computation matched the voltmeter nearly across the range, with the largest difference being roughly 3-4 mV at the midpoint. This is about what you'd anticipate from a 12-bit ADC with a step size of 0.806 mV. The voltmeter has its own tolerance, thus the comparison isn't ideal.

Moving to AIN4 for Activity 2 consisted primarily of altering register values. The SSMUX3 register is the crucial change since it instructs the sequencer which channel to sample. The measurements matched AIN0 at every point, confirming that both channels are receiving the same voltage from the potentiometer wiper.

Conclusion

Both activities worked as planned. Once the pins are properly configured, the TM4C123 ADC is easy to configure at the register level. The 12-bit resolution provides approximately 0.8 mV per step, which is enough for most general-purpose analog measurements. Moving between AIN0 and AIN4 only needs altering the GPIO port clock, the analog pin setup registers, and the SSMUX3 value; the rest of the driver remains unchanged.

Lab Questions

Q1. What is the significance of ADC resolution in this exercise?

Resolution refers to how finely the ADC can discriminate between voltage levels. The TM4C123 has a 12-bit ADC with 4096 potential output values (0-4095). With a 3.3 V reference, each step is around 0.806 mV. That's great for this exercise with a potentiometer; you can sweep smoothly from 0 to 3.3V while seeing the raw value move continually. If it were only 8-bit (256 steps), each step would be roughly 12.9 mV, and the output would change more noticeably as you rotated the pot.

Q2. How does reference voltage affect ADC readings?

The reference voltage determines the top of the ADC's measuring range. The formula is $V_{out} = (\text{raw} / 4095) \times V_{ref}$, therefore adjusting V_{ref} alters what a given raw number represents. If V_{ref} is 3.3V, a raw reading of 2048 equals 1.65V. If V_{ref} was 5V, the identical raw reading of 2048 would equal 2.50V. By default, the TM4C123's reference is set to 3.3V ($V_{REFP} = 3.3V$, $V_{REFN} = GND$), hence any analog input over 3.3V would saturate the ADC at 4095, while any input below 0V would clip to 0.

Q3. What are some possible sources of error in ADC readings?

- Quantization error: inherent rounding to the nearest step ($\pm 0.5 \text{ LSB} = \pm 0.4 \text{ mV}$ at 3.3V/12-bit).
- Noise on the analog input line — long wires or nearby switching circuits can couple interference into the ADC pin.
- Reference voltage instability — if V_{ref} fluctuates slightly the conversion scale shifts.
- ADC offset and gain error — the TM4C123 ADC has a specified offset error of $\pm 2 \text{ LSB}$ typical.
- Temperature drift — the potentiometer's resistance and the ADC's internal characteristics change slightly with temperature.
- Impedance mismatch — if the source driving the ADC pin has too high an output impedance, the sampling capacitor inside the ADC doesn't fully charge before the conversion starts.

Q4. Explain the process of converting an analog signal into a digital value.

The ADC operates in many steps. First, the sample-and-hold circuit quickly connects the input voltage to an internal capacitor before disconnecting it, thus freezing the voltage level for conversion. The sequential approximation register (SAR) logic then takes over, starting with the MSB (is the voltage above or below half of V_{ref} ?), setting that bit, and working its way down through the other 12 bits. After 12 comparisons, it returns a binary value indicating where the input voltage falls in the 0-to- V_{ref} range. The procedure requires a predetermined number of clock cycles (usually 1 μs at 1 MHz sample rate for the TM4C123).

Q5. ADC output range for 3.3V vs 5V reference

For both cases the ADC is 12-bit so the output codes are always 0 to 4095. The difference is what voltage each code maps to:

- 3.3V reference: 0 = 0V, 4095 = 3.3V, resolution = $3.3 / 4095 \approx 0.806$ mV per step.
- 5V reference: 0 = 0V, 4095 = 5.0V, resolution = $5.0 / 4095 \approx 1.221$ mV per step.

The 5V reference gives coarser resolution since the same 4096 steps are spread across a wider voltage range. The TM4C123 ADC input pins are only 3.3V tolerant, so a 5V reference would require using an external voltage divider or a level-shifted reference circuit.

Q6. Sensor with 0–2.5V range, 1 mV sensitivity

Step 1 — Calculate the ADC's voltage resolution at 3.3V reference:

$$\text{Resolution} = V_{\text{ref}} / (2^n - 1) = 3.3 / 4095 \approx 0.000806 \text{ V} = 0.806 \text{ mV}$$

Step 2 — Can the ADC detect a 1 mV change?

The sensor can detect changes as little as 1 mV. The ADC's step size is 0.806 mV. The ADC can detect a 1 mV input change as a 1-2 count variation in the output, as 0.806 mV is less than 1 mV. With normal ADC noise and offset error of ± 2 LSB (approximately ± 1.6 mV), a single 1 mV shift may not result in a consistent reading change. In practice, the ADC is negligible for this sensor.

Step 3 — Two ways to improve accuracy:

- Use a lower reference voltage: if V_{ref} is set to 2.5V (matching the sensor's maximum output), the resolution improves to $2.5 / 4095 \approx 0.610$ mV per step, giving better sensitivity and a more stable reading for 1 mV changes.
- Use hardware averaging: the TM4C123 ADC supports hardware oversampling (2x, 4x, 8x, 16x, 64x). Enabling 64x hardware averaging effectively reduces noise by about a factor of 8, making small voltage changes much more detectable without any software changes.

References

Mazidi, M. A., Chen, S., Naimi, S., & Naimi, S. (2017). TI ARM Peripherals Programming and Interfacing: Programming ARM Cortex-M4 TM4C123G with C. MicroDigitalEd.com.

Texas Instruments. (2014). Tiva TM4C123GH6PM Microcontroller Data Sheet (Rev. E). <https://www.ti.com/lit/ds/spms376e/spms376e.pdf>

Writing Advice Home. University of Toronto.

<https://advice.writing.utoronto.ca/types-of-writing/lab-report/>